



BookShelf App

Project Overview & Product Roadmap

Version 2.0 | April 2026

Platform Android (iOS planned)	Tech Stack Flutter / Dart	Stage Pre-Development
--	-------------------------------------	---------------------------------

1. Project Overview

BookShelf is a mobile-first app built on three product pillars: unified capture, smart organization, and library-first surfacing. It consolidates all the places readers discover books — photos, screenshots, friend recommendations, notes — into one organized visual library. Then, when the user asks "what should I read?", BookShelf intelligently rediscovers books from their own collection rather than pushing generic external recommendations.

The idea was born from a real problem: a person who regularly photographs book covers in bookstores, screenshots recommendations from Instagram, saves titles to Pinterest boards, and jots notes in spreadsheets — only to have all of these scattered across different apps and impossible to revisit. She had tried Goodreads and Notion, but both felt like admin work and were abandoned. She is not alone — this fragmented discovery behavior is extremely common, and no app currently unifies it well.

Three Product Pillars

1. UNIFIED CAPTURE — Bring photos, screenshots, manual entries, and recommendations into one place.
2. SMART ORGANIZATION — Sort, tag, and auto-categorize so the library stays useful as it grows.
3. LIBRARY-FIRST SURFACING — Answer "what should I read?" by rediscovering books you already saved, not generic external recommendations.

2. The Problem We Are Solving

Readers face two interconnected problems: fragmented discovery and the saved-list paradox.

Problem 1: Fragmented Discovery

Books get discovered everywhere — bookstores, Instagram, friend recommendations, articles, podcasts — and end up scattered across camera rolls, screenshots, Pinterest boards, notes apps, and spreadsheets. There is no unified view of what you actually want to read.

Problem 2: The Saved-List Paradox

Even when readers do save books somewhere, they rarely come back to read them. The same problem plagues Pocket articles, Instagram saves, and YouTube "Watch Later" — saving feels productive, but without smart resurfacing, the list becomes a graveyard. Margot's exact words: "browsing my list kills the mood quickly because I get overwhelmed with choice."

Existing solutions fall short on both fronts:

- Fragmented capture: Saving to camera roll, Instagram, Pinterest, or notes scatters books across apps. Manual consolidation is tedious work that nobody actually does.
- Too much friction: Apps like Goodreads and StoryGraph require searching, manual entry, logging, and maintaining a social profile. Casual readers abandon them quickly.
- Poor execution: Apps like ScanMyBook offer cover scanning but treat it as a utility, not a unified library, with no thoughtful resurfacing of what you've saved.
- Generic recommendations: LLMs and Google can suggest "books like X" but cannot see your library. They push you toward more books to buy, not the books you already wanted to read.

Key Insight from User Research (Margot, target user)

"I have books in my Photos app, Instagram saved posts, a Pinterest board, an old Goodreads account, and a Notion dashboard I haven't touched in years."

"Browsing my list kills the mood quickly because I get overwhelmed with choice."

"I turn off push notifications on almost every app — I don't want Duolingo-style guilt trips."

"When I open Google Photos and see housekeeping prompts at the top, that feels natural and helpful."

3. Target Audience

Segment	Description	Size / Notes
Primary: Casual Discoverers	Readers who collect book recommendations but struggle to act on them. Photograph covers, save Instagram posts, make Pinterest boards.	Underserved by all existing apps. High volume.
Secondary: Avid Readers	Readers who want a beautiful visual bookshelf without the social overhead of Goodreads. Want to grow a "read" list.	Currently use StoryGraph or Goodreads reluctantly.
Tertiary: Gift Givers & Shoppers	People photographing books in stores for later purchase or as gift ideas.	Adjacent use case; low lift to support.

4. Competitive Landscape

The book tracking space is crowded but fragmented. Many apps exist, but none have combined visual-first cover scanning, private-by-default library management, and mood-based discovery in a polished, well-executed product. The market is underserved, not empty.

Direct Competitors

Apps that already scan book covers using AI/ML recognition:

App	Status & Notes
ScanMyBook	Closest direct competitor. Scans covers with AI, builds collections, has friend activity. Available on iOS and Google Play (Oct 2025). Major weaknesses: forced paid trial frustrates users, appears to be solo developer project with minimal traction and reviews. UX is utility-first, not visual-first.
Shelf Scan (iOS)	AI-scans entire bookshelves from one photo. Focused on discovery (ratings, highlight top-rated books) rather than personal library management. Not a TBR tracker.
Book Scanners (Google Play)	Photo-based book identification. Minimal reviews, unclear quality. Appears to be low-effort project.
Bookshelf Analyzer	Web-based GPT-4 tool (not an app). Identifies books from shelf photos. Free but requires manual upload, no mobile-first experience.

Established Players (No Cover Scanning)

Major apps with barcode scanning only or no visual capture at all:

App	Strengths & Weaknesses
Goodreads	Barcode and cover scan (hidden feature since 2017). Massively social, not private. Users report it feels like admin work. Dominant player but widely disliked by casual readers.
StoryGraph	Mood-based recommendations and progress tracking. Barcode scan only. Strong on analytics, weak on visual UX. No cover photo recognition.
Reading List (iOS)	Private by default, on-device storage, barcode scan. Clean UX but barcode-only limits discoverability. No mood filtering or reminders.
Basmo	OCR scanning of pages, reading habit tracking, push notification reminders. Focuses on reflection and journaling, not discovery pipeline.
Candl, Hardcover, Literal	Privacy-first or social-first variations. None have visual cover capture or mood-based browsing as core features.

Market Reality: The Competition Exists, But Execution Is Weak

ScanMyBook proves the tech works and there is demand — but the product is poorly executed and has no visible traction.

Goodreads has cover scanning but it is buried and not marketed. Most users do not know it exists.

No app combines visual-first UX + mood rescue + in-app housekeeping nudges in a polished, well-designed product.

Our Competitive Advantage

BookShelf is not the first to scan covers. It is the first to solve the saved-list paradox — turning a graveyard of saved books into something you actually read from.

- Library-first surfacing: When users ask "what should I read?", we answer with their own library first. LLMs and Google can't do this — they don't know what you've saved. ScanMyBook recommends generic similar books, not from your library.
- Tinder-style decision flow: 2-3 simple questions surface ONE book at a time, solving the "overwhelmed by choice" problem that kills every other TBR app.
- Consolidation moat: We unify photos, screenshots, and manual entries into one library. ScanMyBook only does camera capture. Goodreads requires manual search. We bring it all together.
- Google Photos-style housekeeping prompts: In-app nudges on open, not guilt-tripping push notifications. This is genuinely novel.
- Visual-first design: The library should feel like a beautiful personal bookshelf, not a database or spreadsheet.

- Pure discovery focus: No progress tracking guilt. No social performance anxiety. Just: discover it, save it, rediscover it when you are ready.
- Polish and execution: ScanMyBook exists but looks like a solo side project. There is room for a well-built, well-designed version.

5. App Features

Features are organized around the three product pillars. The MVP delivers a basic version of each pillar; later phases deepen the smart organization and surfacing capabilities.

MVP — Phase 1 (All Three Pillars, Basic)

Pillar 1: Unified Capture

- Camera capture with AI cover recognition: photograph any book cover and match to Google Books / Open Library metadata
- Manual entry: add books by title or author when you can't photograph them (verbal recommendations, books from articles)
- Photo library import: upload existing book photos and screenshots from camera roll

Pillar 2: Smart Organization

- Visual library: grid or shelf view of saved books with cover art — unified view regardless of source
- Manual shelves and tags: create custom categories like "Beach reads" or "Recommended by friends"
- Basic auto-sort: by author, date added, title, or genre (pulled from book metadata)

Pillar 3: Library-First Surfacing

- "What should I read?" flow: 2-3 simple questions (mood, time available, fiction/nonfiction) surface ONE book from your library at a time — Tinder-style, not overwhelming list-style
- Book detail page: author bio, series info, similar books
- In-app housekeeping prompts: "Did you end up reading this?" on app open, no push notifications

Phase 2 — Smarter Organization & Surfacing

- AI auto-tagging: automatically tag books with mood, genre, and themes using cover art and metadata — solves the cold-start tagging problem
- Mood-based browsing: filter your library by mood (cozy, intense, funny, escapist) using AI-generated tags

- Smart auto-categorization: AI sorts books into shelves based on cover and content
- Screenshot text recognition: detect book titles in screenshots using OCR, even when no cover is visible
- Series awareness: when you save Book 1, app prompts about saving the rest of the series

Phase 3 — External Recommendations & Social

- External recommendations: "You've read everything in your library — here are 5 new books based on your taste" (only when library is exhausted or user explicitly requests)
- Goodreads / StoryGraph import: pull in existing TBR lists from other platforms
- Shareable lists: optionally share a curated reading list with friends (view-only link)
- BookShelf Pro subscription: advanced filters, unlimited shelves, export, themes
- Affiliate links: optional "buy this book" button with Amazon / Bookshop.org affiliate revenue

Cost Management for AI Features:

AI is used at ingest time (one-time tagging when a book is added), not at recommendation time. Recommendations are served from pre-computed tags and embeddings as inexpensive database operations. Open-ended natural-language queries are reserved for Pro-tier users with daily limits, ensuring AI costs scale with revenue rather than free-tier usage.

6. Technical Approach

Flutter / Dart was chosen as the primary framework because it enables a single codebase for both Android (primary) and iOS (future), aligning with the team's existing skills and the phased platform rollout.

Layer	Technology / Approach
Frontend	Flutter / Dart — cross-platform, single codebase
Book Data API	Google Books API (primary), Open Library (fallback)
Cover Recognition	Google ML Kit on-device vision + Books API lookup
Backend / Storage	Firebase (Auth, Firestore, Cloud Storage) — scalable and Flutter-native
Notifications	In-app state-based prompts (no push notifications in MVP)
Monetization	Google Play Billing for Pro subscription; affiliate deep links

7. Monetization Strategy

BookShelf will follow a freemium model. The free tier must be genuinely useful to drive organic growth; the Pro tier adds power-user features without paywalling the core experience.

Tier	What's Included
Free	Full camera capture, visual library (up to 100 books), basic book details, in-app prompts, 3 shelves
Pro (\$2.99/mo or \$19.99/yr)	Unlimited books, unlimited shelves, mood filters, auto-categorization, CSV export, custom themes
Affiliate Revenue	Optional "Where to Buy" button on book detail pages — passive income from purchases
Future: API licensing	License book-cover-to-metadata API to other developers once validated at scale

8. Product Roadmap

Phase	Timeline	Milestones
Discovery	Now — May 2026	Finalize feature list with Margot. Define MVP scope. Set up Flutter project and Firebase. Evaluate Google Books API coverage.
MVP Build	May — July 2026	Camera capture + cover recognition. Visual library UI. Book detail pages. In-app housekeeping prompts. Internal beta with Margot.
Beta	Aug — Sep 2026	Expand beta to 20–50 users. Mood tagging. Manual shelves. Collect feedback. Iterate on UX.
Launch v1.0	Oct 2026	Google Play Store launch. Free + Pro tiers live. Affiliate links. App Store submission begins.
Phase 2	Q1 2027	iOS launch. Auto-categorization AI. Screenshot import. Social sharing (view-only lists).

9. Next Steps — First Meeting Agenda

The goal of the first meeting is to align on MVP scope, validate assumptions with Margot, and assign initial responsibilities. Suggested agenda:

#	Agenda Item
1	Walk through Margot's responses and confirm the core problem statement
	Confirmed. Possibility of export / import to potentially work with or steal users from other apps.
2	Define what "done" looks like for the MVP — what is the minimum that would make Margot use this daily?
	Upload photos from camera roll, it understands the title, date taken, and author and genre, and it puts it into a table or visual board with all meta data (there is a row of sponsored books at the top and then her visual library). That table or board or data has side bars or info could then easily be exported. AND we have import from other apps
3	Discuss the mood-based browsing concept — what does it look like in practice?
	Mood based browsing is fine, but don't want to scroll endlessly. Storygraph filters by mood (not sure how that is determined)
	Possibly integrate a quiz to find saved books by mood ONLY 2 options.
4	Review the in-app housekeeping prompt design — how often, what triggers it, what does it say?
	Allow notifications, but easily turn off.
	The notifications should ask: "Do you still want this book in your library" (time triggered) "Do you want to rate this book"
5	Agree on tech stack: Flutter and Dart
	Approved
6	Set up a shared repo and project board (GitHub + linear or Notion)
	Completed
7	Assign owners: Who builds? Who tests? Who is the "user voice" (Margot)?
	Bode and Richie Build and Test - Margot is user voice
8	Set a date for the next check-in
	TBD

Open Questions to Resolve

What 2-3 questions should the "What should I read?" flow ask? (mood, time, fiction/nonfiction?)

How do we handle the cold-start tagging problem in MVP without AI auto-tagging?

Should MVP include screenshot OCR, or is photo upload + manual entry enough?

How does the app handle books it cannot recognize from the cover photo?

What is the app name? (To be decided together)

Should we test ScanMyBook ourselves to understand where it falls short?

What privacy controls does Margot want over her library data?

10. Individual Goals

Richie:

- Goals:
 - Wants to be net positive
 - Have something published on an app store that can be reviewed by people and used
 - Have built something that would personally use and find value in
- Skills / wants to do:
 - Code drafts, UX / UI design
 - General project guidance
 - Marketing

Bode:

- Goals:
 - Experience building an app that people use
 - Make money from it
 - Soft completion goal time of January 2027
- Skills / wants to do:
 - Coding, learn API's

Margot:

- Wants:
 - Take existing photos from photo library, upload them and organize them
 - .csv exports from app to have option to go app to app
- Skills / wants to do:
 - Beta Tester, Target User, Insight into Workflow

11. To-Do List

Privacy policy: Explaining we're using the data

12. Ideal Workflow v1

Margot has photos in her camera roll of a bunch of book covers, go into the app, the app imports the photos, asks if the titles and authors are correct, you approve (individually by uncertainty threshold if the image is blurry or approve all button), If the app can't tell what the text in the image says: It shows the book it thinks it is, the image you submitted, you say yes or no, if no, requires manual input.

It populates a dashboard with all of the book covers, you can tap on the book and a sidebar provides info about it like places you can buy it, summary, general rating, genre, tags, folders, filters. Be able to rate and review for yourself but not needed to share with friends.

13. Next Steps

Step 1: Set up the foundation - First session together, ~2 hours? In order:

1. Create a GitHub repo — private, name it something like `bookshelf-app`
2. Create a new Flutter project inside it (`flutter create bookshelf`)
3. Set up a Firebase project at firebase.google.com — enable Firestore, Firebase Auth, and Cloud Storage
4. Connect Flutter to Firebase using the FlutterFire CLI (`flutterfire configure`)
5. Add collaborators on both GitHub and Firebase for full visibility

By end of Step 1 we have: A blank Flutter app that talks to Firebase, on GitHub, that both of us can pull down.

Step 2: Get it running on both phones Before writing a single feature, get the blank app running on actual devices. Sounds trivial but catches setup issues early.

- Bode runs it on his Android device via USB debug
- Run it on a second device (Android or iPhone — Flutter handles both)
- If anything breaks here, fix it now before any real code exists

By end of Step 2 we have: The blank app running on two real devices.

Step 3: Homework to do in parallel While Bode is setting up infrastructure:

1. Get a Google Books API key — go to console.cloud.google.com, create a project, enable the Books API, generate a key. This is what will identify books from photos. Send the key to Bode.
2. Gather Margot's test photos — ask her to send 20-30 actual book cover photos from her camera roll, including blurry or awkward ones. The messier the better.
3. Make a simple spreadsheet of those 20-30 books — title, author, what the photo looks like. This becomes the ground truth to measure how well AI recognition is working.
4. Sketch the approval screen — the most important UX moment in the MVP. When the app imports photos and isn't sure about a title, what does that screen look like? Sketch it

on paper or Figma. Specifically: what does the uncertainty threshold look like visually?
How does "approve all" vs individual approval feel?

Step 4: Build the single most important feature first - Don't build the dashboard yet. Don't build the sidebar yet. Build one thing:

Photo import → book identification → show a result

Full flow:

1. User picks a photo from camera roll
2. App sends image to Google Cloud Vision API (to extract text from cover)
3. App takes that text and queries Google Books API
4. App displays: cover image, identified title, author, confidence level

No database yet. No UI polish. Just: does the recognition actually work?

This is the proof of concept. If this works reliably on Margot's actual photos, we have a real app. If it doesn't, we need to know now before building everything else on top of it.

Step 5: Test it on Margot's photos Run all 30 photos through the identification flow. Track:

- How many did it get exactly right?
- How many were close but wrong?
- How many did it completely fail on?
- Which types of photos fail most (blurry, angled, partial covers)?

Need roughly 80%+ accuracy on clean photos before building further. If we're at 50%, the core product doesn't work yet and that needs to be solved before anything else.

Step 6: Build the MVP in this order Once recognition is proven, build in this sequence — each step is testable on its own:

1. Save identified books to Firestore (the database layer)
2. The approval/confirmation screen (individual + approve all)
3. The visual dashboard (grid of covers)
4. The book detail sidebar (tap a cover, see metadata)
5. Basic tagging and filtering
6. The mood quiz → 2 recommendations flow
7. The two time-triggered in-app notifications
8. CSV import